

Order Management Full Stack

A full-stack order management system built with **ASP.NET Core (.NET 9)** for the backend and **Blazor Web App** for the frontend.

Designed using clean architecture principles with a focus on scalability, maintainability, and real-world practices.

Features

- Full CRUD operations for Clients, Suppliers, Goods, and Orders
 - Relational data model with proper entity relationships
 - DTO pattern for clean API contracts
 - Repository + Service layer architecture
 - Entity Framework Core with SQL Server
 - Automatic database creation and seeding
 - OpenAPI + Swagger UI for backend testing
 - Blazor Web App frontend with interactive Server rendering
 - Generic service pattern for reusable frontend data access
-

Architecture

Backend

Controllers → Services → Repositories → DbContext → Database

- **Controllers:** Handle HTTP requests
- **Services:** Business logic
- **Repositories:** Data access
- **DTOs:** Separate API models from database entities

Frontend

Blazor Pages → Services (GenericService<T>) → HttpClient → Backend API

- **Pages:** Razor components for List, Read, Create, Edit, Delete per entity
 - **Services:** Generic HTTP service layer with typed models
 - **Models:** Mirror of backend DTOs for JSON deserialization
-

🔧 Tech Stack

Layer	Technology
Backend	ASP.NET Core (.NET 9), C#
Frontend	Blazor Web App (.NET 9), C#
Database	SQL Server / LocalDB
ORM	Entity Framework Core
API Docs	OpenAPI / Swagger

📁 Project Structure

```
OrderManagementFull/  
├── OrderSystem/                                # Backend API  
│   ├── Controllers/  
│   ├── Services/  
│   ├── Repositories/  
│   ├── Models/  
│   ├── DTOs/  
│   └── Data/  
└── OrderSystem.UI/                            # Blazor Frontend  
    ├── Components/  
    │   ├── Pages/  
    │   │   ├── Goods/  
    │   │   ├── Clients/  
    │   │   ├── Suppliers/  
    │   │   └── Orders/  
    │   └── Layout/  
    ├── Models/  
    ├── Services/  
    └── Program.cs
```

▶ Getting Started

Prerequisites

- .NET 9 SDK

- SQL Server or LocalDB
- Visual Studio 2022 (recommended)

1. Clone the repository

```
bash

git clone https://github.com/ivanescad-london/OrderManagementFull.git
cd OrderManagementFull
```

2. Configure the database

Update `appsettings.json` in the backend project if needed:

```
json

"ConnectionStrings": {
  "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=OrderSystemDb;Truste
}
```

3. Run the application

Set both projects as startup projects in Visual Studio:

Right-click Solution → **Set Startup Projects** → set both to **Start**

Or run separately:

```
bash

# Backend
cd OrderSystem
dotnet restore
dotnet ef database update
dotnet run

# Frontend
cd OrderSystem.UI
dotnet run
```

4. Open the applications

- **Blazor Frontend:** `https://localhost:5001`
 - **Swagger UI:** `https://localhost:7296/swagger`
-

API Endpoints

All four controllers follow the same endpoint pattern:

Method	Route	Description
GET	<code>/api/{entity}/GetAll</code>	Get all records
GET	<code>/api/{entity}/Read/{id}</code>	Get by ID
POST	<code>/api/{entity}/Create</code>	Create new record
PUT	<code>/api/{entity}/Update/{id}</code>	Update existing record
DELETE	<code>/api/{entity}/Delete/{id}</code>	Delete record

Example: Create a Good

```
http
POST /api/Goods/Create
{
  "name": "Widget A",
  "price": 9.99
}
```

Example: Create an Order

```
http
POST /api/Orders/Create
{
  "orderDate": "2026-04-08",
  "clientId": 1,
  "supplierId": 1,
  "goodsId": 1,
  "quantity": 2
}
```

Seed Data

On first run, the application automatically creates:

- 4 Clients
- 3 Suppliers

- 3 Goods
- 6 Orders

This allows immediate testing without manual setup.

Purpose

This project demonstrates:

- Building RESTful APIs with ASP.NET Core
 - Building interactive UIs with Blazor Web App (C# instead of JavaScript)
 - Applying clean architecture patterns across frontend and backend
 - Generic programming patterns for reusable service layers
 - Working with relational data using EF Core
 - Connecting a Blazor frontend to a REST API via HttpClient
 - Designing scalable full-stack systems
-

Future Improvements

- Pagination and filtering
 - JWT authentication
 - Unit testing
 - Docker support
 - Form validation improvements
 - Better error handling and user feedback
-

Author

Ivan Simeonov

 <https://github.com/ivanescad-london>